
SE4020 – MOBILE APPLICATION DESIGN AND DEVELOPMENT
NURSERYCONNECT macOS APPLICATION
ASSIGNMENT 2 REPORT PART A

B.Sc. (Hons) in Information Technology

Bamunusinghe S A N

IT22515612

Group Y4.S2.WE.SE.02.02

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

June 2026

DECLARATION

I declare that this is my own work and this assignment does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning. To the best of my knowledge and belief it does not contain any material previously published or written by another person except where acknowledgement is made in the text.

Name	IT Number	Signature	Date
Bamunusinghe S A N	IT22515612		02 June 2026

AI tool assistance was used during development as permitted under Submission Guideline 3. All AI-generated code was reviewed, understood, tested, and modified by the author. Full details are documented in Appendix B.

ABSTRACT

This report documents the design, development, and regulatory analysis of the Assignment 2 Part A extension of the NurseryConnect mobile application for SE4020 Mobile Application Design and Development at SLIIT. The application targets macOS and adopts a new user role the Setting Manager (Nursery Manager) expanding beyond the Keyworker role implemented in Assignment 1.

The macOS application, CareBridge-macOS, is built in SwiftUI using the MVVM architectural pattern and comprises 83 Swift files across 19,680 lines. The Setting Manager dashboard implements a three-column NavigationSplitView with macOS-native toolbar commands, keyboard shortcuts, multi-window support, a Menu Bar Extra, and drag-and-drop. Three advanced Apple frameworks are integrated NaturalLanguage (NLTagger for sentiment analysis, NLPTokenizer for EYFS keyword extraction, NLPLanguageRecognizer for Ofsted language compliance), Swift Charts (LineMark + AreaMark wellbeing trend chart BarMark EYFS coverage chart SectorMark mood distribution chart), and PDFKit (paginated Ofsted-ready report generation with NSSavePanel export).

The new Setting Manager role introduces AI-powered Child Development Analytics, an Incident Review Queue with countersignature workflow, EYFS Coverage Heatmap, Wellbeing Alert System, and Report Generation with PDF export. A dual-role architecture allows both Keyworker (from Assignment 1) and Setting Manager workflows within a single macOS application, with a Role Selection screen on launch.

Three distinct AI-generated UI mockup variations were produced using Claude Artifacts (Anthropic) and are critically evaluated against macOS HIG conventions and the NurseryConnect childcare context. Regulatory compliance analysis covers UK GDPR, EYFS 2024, Ofsted EIF, and Children Act 1989 from the Setting Manager perspective.

Keywords SwiftUI, macOS, NurseryConnect, Setting Manager, NaturalLanguage, Swift Charts, PDFKit, MVVM, @Observable, NavigationSplitView, MenuBarExtra, Multi-window, EYFS, AI Sentiment Analysis, Ofsted Reports, Wellbeing Analytics

TABLE OF CONTENTS

DECLARATION	2
ABSTRACT	3
TABLE OF CONTENTS	4
LIST OF ABBREVIATIONS	7
CHAPTER 1 INTRODUCTION	8
1.1 Background and Context	8
1.2 Chosen Platform macOS	8
1.3 Chosen User Role Setting Manager	8
1.4 New Features Added	9
1.5 Report Structure	10
CHAPTER 2 PLATFORM SELECTION AND EXTENSION JUSTIFICATION	11
2.1 Justification for macOS over iPadOS	11
2.2 What is New vs. Carried Forward from Assignment 1	12
CHAPTER 3 APPLICATION DESIGN	13
3.1 Architecture MVVM with @Observable Service Layer	13
3.2 macOS Navigation Design NavigationSplitView Three-Column Layout	13
3.3 AI-Driven UI Design Process	14
3.3.1 Tool Selection Rationale	14
3.3.2 Mockup Variation 1 Data-Dense Analytics Dashboard	14
3.3.3 Mockup Variation 2 Card-Based Manager Overview	15
3.3.4 Mockup Variation 3 macOS-Native Three-Column Layout (Selected)	15
3.3.5 Final Design Decision and Synthesis	16
3.4 Design System Continuity	16
3.5 HCI Principles Applied (macOS Context)	16

CHAPTER 4 IMPLEMENTATION	18
4.1 Technology Stack	18
4.2 Codebase Overview	18
4.3 Code Snippet 1 NLPAnalysisService Sentiment Analysis	19
4.4 Code Snippet 2 CareBridgeApp macOS Multi-Window Architecture	20
4.5 Code Snippet 3 Swift Charts WellbeingTrendChartView	22
4.6 Code Snippet 4 PDFKit Report Generation	23
4.7 Advanced SwiftUI Concepts Summary	24
CHAPTER 5 ADVANCED LIBRARY INTEGRATION.....	26
5.1 NaturalLanguage Framework.....	26
5.1.1 Integration Overview.....	26
5.1.2 Wellbeing Score Computation Algorithm.....	26
5.2 Swift Charts Framework	27
5.2.1 Three Chart Types Implemented.....	27
5.3 PDFKit Framework	27
5.4 Genuine Value Assessment.....	28
CHAPTER 6 TESTING AND DEBUGGING.....	29
6.1 Testing Strategy.....	29
6.2 XCTest Unit Tests CareBridgeTests.swift	29
6.3 Runtime Diagnostics Dashboard (DiagnosticsDashboardView)	29
6.4 Error Handling Strategy	30
6.5 Debugging Techniques.....	30
CHAPTER 7 REGULATORY COMPLIANCE ANALYSIS.....	32
7.1 Introduction	32
7.2 UK GDPR Setting Manager Obligations	32
7.3 EYFS Framework 2024 Setting Manager Obligations	32

7.4 Children Act 1989 Setting Manager Obligations	33
7.5 Ofsted EIF 2023 Inspection Readiness	34
CHAPTER 8 CHALLENGES AND REFLECTIONS	35
8.1 Technical Challenges	35
8.1.1 Async NL Analysis Without Blocking the Main Thread	35
8.1.2 PDFKit HTML Rendering via NSPrintOperation.....	35
8.1.3 Multi-window State Coordination.....	35
8.1.4 NLLanguageRecognizer Confidence Threshold	35
8.2 Lessons Learned	36
REFERENCES.....	37
APPENDIX A AI-GENERATED UI MOCKUP VARIATIONS	39
A.1 AI Tool Used Claude Artifacts (Anthropic).....	39
A.2 Mockup Variation 1 Data-Dense Analytics Dashboard	39
A.3 Mockup Variation 2 Card-Based Manager Overview.....	40
A.4 Mockup Variation 3 macOS-Native Three-Column Layout (Selected).....	41
A.5 Final Design Synthesis	42
APPENDIX B AI TOOL USAGE LOG	43

LIST OF ABBREVIATIONS

Abbreviation	Meaning
EYFS	Early Years Foundation Stage
GDPR	General Data Protection Regulation
MVVM	Model-View-ViewModel
MVP	Minimum Viable Product
RIDDOR	Reporting of Injuries, Diseases and Dangerous Occurrences Regulations
HIG	Human Interface Guidelines
WCAG	Web Content Accessibility Guidelines
SLIIT	Sri Lanka Institute of Information Technology
NLP	Natural Language Processing
NER	Named Entity Recognition
EYFS	Early Years Foundation Stage
DSL	Designated Safeguarding Lead
DPO	Data Protection Officer
EIF	Education Inspection Framework
OSLog	Apple Unified Logging System
AI	Artificial Intelligence
PDF	Portable Document Format
RBAC	Role-Based Access Control

CHAPTER 1 INTRODUCTION

1.1 Background and Context

Assignment 1 delivered a SwiftUI iOS MVP for the NurseryConnect platform from the perspective of the Keyworker role, implementing the Daily Diary & Activity Monitoring and Incident Management features for Little Stars Nursery & Daycare. The full Assignment 1 codebase is available at <https://github.com/ArunaluB/carebridge-ios.git>.

Assignment 2 Part A extends this foundation by targeting macOS and adopting a complementary user role the Setting Manager (Nursery Manager). The macOS platform was selected as it is the primary desktop platform used in UK nursery office environments where managers perform administrative oversight, generate Ofsted-ready reports, review incident queues, and analyse cross-child wellbeing data tasks requiring the multi-window, multi-column, and keyboard-driven interaction model that macOS natively provides.

1.2 Chosen Platform macOS

macOS was selected over iPadOS for the following reasons. The Setting Manager's role is primarily office-based they review data across all children, countersign incidents, generate PDF reports for Ofsted inspections, and analyse AI-computed wellbeing scores all tasks that benefit from a large screen, keyboard shortcuts, and multi-window workflows. The three-column NavigationSplitView layout, macOS Menu Bar commands, keyboard shortcuts, drag-and-drop, and Menu Bar Extra are platform-native affordances that genuinely serve the Setting Manager's operational context and cannot be replicated on iOS or iPadOS with the same ergonomic fidelity.

1.3 Chosen User Role Setting Manager

The Setting Manager (Nursery Manager / Room Leader) role was selected as the complementary Assignment 2 role to the Assignment 1 Keyworker. Under the EYFS statutory framework, the Setting Manager is responsible for Ofsted inspection readiness, countersigning all incident reports before parent notification, overseeing all children's developmental progress across rooms, managing keyworker assignments, and generating weekly and monthly compliance reports. This role has direct access to data aggregated across all children contrasting

with the Keyworker's assignedChildrenIds-scoped view implemented in Assignment 1 providing both a technically and contextually distinct extension.

1.4 New Features Added

The following features were added in Assignment 2 Part A, which are entirely new and not present in the Assignment 1 iOS application

- AI-Powered Child Development Analytics NaturalLanguage framework sentiment analysis, keyword extraction, and wellbeing score computation per child
- EYFS Coverage Dashboard Swift Charts BarMark chart showing activity coverage across 7 EYFS learning areas
- Wellbeing Trend Chart Swift Charts LineMark + AreaMark 14-day rolling wellbeing score chart with threshold annotations
- Mood Distribution Chart Swift Charts SectorMark (pie chart) showing mood rating distribution
- Incident Review Queue Setting Manager countersignature workflow (distinct from Keyworker incident submission in Assignment 1)
- PDF Report Generation PDFKit-based Ofsted-ready report generation (Weekly Summary, Incident Report, EYFS Coverage) with NSSavePanel export
- Setting Manager Dashboard NavigationSplitView three-column layout with metrics, wellbeing alerts, and attention list
- Role Selection Screen Dual-role architecture supporting both Keyworker (Assignment 1) and Setting Manager (Assignment 2) within a single macOS application
- Menu Bar Extra Status bar app for quick access to the dashboard without opening the main window
- Multi-window Support WindowGroup(for) enabling individual child profile windows opened in new macOS windows
- Diagnostics Dashboard Separate window (WindowGroup id) for running automated diagnostic tests

1.5 Report Structure

Chapter 2 justifies the platform and role selection. Chapter 3 covers application design including the AI-driven UI mockup process. Chapter 4 documents implementation with code snippets. Chapter 5 details advanced library integration. Chapter 6 addresses testing and debugging. Chapter 7 presents the regulatory compliance analysis. Chapter 8 reflects on challenges. Appendix A contains AI mockup variations. Appendix B documents AI tool usage.

CHAPTER 2 PLATFORM SELECTION AND EXTENSION

JUSTIFICATION

2.1 Justification for macOS over iPadOS

Factor	macOS Selection Rationale	iPadOS Alternative Assessment
Primary use context	Office-based administrative oversight keyboard-driven large-screen multi-column data review	Mobile/portable touch-first less suited to data-heavy management workflows
NavigationSplitView	Three-column layout is the macOS standard for manager dashboards sidebar + content list + detail panel	Two-column split view typical on iPad three columns less ergonomic
PDF export	NSSavePanel + macOS file system integration is native to macOS familiar to managers exporting Ofsted reports	ShareSheet on iPadOS is less direct for file system access
Multi-window	WindowGroup(for) opens per-child profile windows simultaneously critical for manager comparing two children	iPadOS multi-window is less common and not keyboard-driven
Menu Bar Extra	MenuBarExtra is macOS-only enables always-visible status monitoring without opening the full app	No equivalent concept on iPadOS
Keyboard shortcuts	CommandMenu +. keyboardShortcut for all primary actions managers work efficiently with keyboard	iPadOS keyboard shortcuts are secondary to touch
Drag and drop	onDrag/onDrop for dragging child cards to open profile windows desktop native interaction	Touch drag is different in semantics and less precise for data operations

2.2 What is New vs. Carried Forward from Assignment 1

Component	Status	Notes
Models.swift, SampleData.swift, DataManager.swift	Carried forward (shared)	Assignment 1 data models are fully reused the macOS app shares the same DataManager and UserDefaults persistence layer
Keyworker views (DailyDiaryView, IncidentListView, etc.)	Carried forward (extended)	All Keyworker views are recompiled for macOS target with macOS-appropriate adaptations (no FAB, no tab bar)
SettingManagerDashboardViewModel	New Assignment 2	Orchestrates NL analysis, aggregates metrics, manages three-column selection state
NLAnalysisService	New Assignment 2	Full NaturalLanguage framework integration sentiment, keywords, location NER, language detection, wellbeing score
ManagerDataService	New Assignment 2	@Observable data service bridging DataManager with Setting Manager-specific queries
Swift Charts views (3 chart types)	New Assignment 2	WellbeingTrendChartView, EYFSCoverageBarChartView, MoodDistributionChartView
PDFKit report generation	New Assignment 2	PDFGenerator + ReportViewModel + PDFPreviewView with NSSavePanel
Multi-window + MenuBarExtra	New Assignment 2	macOS-exclusive WindowGroup(for), MenuBarExtra, CommandMenu, Settings scene
ChildDevelopmentDetailView	New Assignment 2	Core AI analytics view per child wellbeing score ring, charts, observation feed
DiagnosticsDashboardView	New Assignment 2	Separate diagnostics window with automated test runner for testing evidence
ManagerModels.swift	New Assignment 2	New data models SettingManagerProfile, ChildWellbeingAlert, SentimentAnalysisResult, WellbeingTrend
RoleSelectionView	New Assignment 2	Dual-role launch screen enabling Keyworker/Manager selection

CHAPTER 3 APPLICATION DESIGN

3.1 Architecture MVVM with @Observable Service Layer

The macOS application follows the same MVVM architectural pattern established in Assignment 1, extended with a dedicated ManagerDataService that bridges the shared DataManager with Setting Manager-specific queries. All ViewModels and Services use Swift 5.9's @Observable macro. The architecture extends naturally to the macOS context

Layer	Responsibility	Key Files (Lines)
Model	Codable structs 16 enums (Assignment 1) + 5 manager-specific types	Models.swift (439), ManagerModels.swift (331), Constants.swift (492)
Service	@Observable singletons NL analysis data aggregation	NLAnalysisService (380), ManagerDataService (330+), DataManager (shared)
ViewModel	Dashboard metrics, AI analysis orchestration, report generation	SettingManagerDashboardViewModel (330+), ChildAnalyticsViewModel, ReportViewModel
View	Declarative SwiftUI macOS views three-column layout	ContentView (NavigationSplitView), OverviewDashboardView, ChildDevelopmentDetailView (core new feature)
Component	Reusable macOS-adapted components	MetricCardView, WellbeingScoreRing, StatusBadge, AlertCardView, SentimentBadge
Utility	Pure static helpers OSLog integration	AppError, AppLogger (OSLog), FormValidator, HapticManager (NSHapticFeedbackManager)

3.2 macOS Navigation Design NavigationSplitView Three-Column Layout

The Setting Manager dashboard uses a three-column NavigationSplitView the standard macOS design pattern for content-rich administrative applications (analogous to Mail, Finder, and Xcode). Each column has a defined purpose

Column	Component	Width	Content
Column 1 Sidebar	SidebarView	200–260pt	App branding, navigation sections (Dashboard, Management, Tools), badge counts, manager profile card
Column 2 Content List	ContentListView	280–400pt	Context-sensitive children list, incident queue, AI insights list, report controls, settings
Column 3 Detail	DetailColumnView	Remaining	Primary content OverviewDashboardView, ChildDevelopmentDetailView, IncidentReviewDetailView, PDFPreviewView

3.3 AI-Driven UI Design Process

3.3.1 Tool Selection Rationale

Claude Artifacts (Anthropic) was selected as the AI-powered UI mockup generation tool for the following reasons it generates interactive HTML/SVG prototypes that can be rendered and tested in the browser not merely static images enabling evaluation of interaction flow, not just visual appearance. Its deep understanding of SwiftUI conventions and macOS HIG guidelines produces mockups that are directly implementable rather than requiring significant translation effort. The conversational refinement capability allows iterative generation based on specific feedback (e.g., 'make the sidebar narrower and add badge counts').

Three distinct mockup variations were generated, each exploring a different design philosophy for the Setting Manager dashboard. The specific prompts and responses are documented in Appendix B.

3.3.2 Mockup Variation 1 Data-Dense Analytics Dashboard

Variation 1 explored a data-dense design inspired by enterprise analytics dashboards (Grafana, Linear). The dashboard prioritised quantitative data visibility four metric cards at the top (Children In, Pending Incidents, Wellbeing Flags, EYFS Coverage %), followed immediately by full-width line charts, and a compact incident list. The colour palette was dark navy with teal accents.

Critical Evaluation The data density is appropriate for the Setting Manager's need to quickly assess nursery status across many children. However, the dark theme conflicted with the established NurseryConnect light/warm colour palette (ncBackgroundLight #FAFAF8, ncPrimary #4ECDC4) designed to evoke a professional childcare environment. The analytics-first layout also deprioritised the incident queue which is a compliance-critical priority for the Setting Manager role under EYFS same-day notification requirements. The layout was accepted for the metric card arrangement but rejected for the dark theme and chart-first hierarchy.

3.3.3 Mockup Variation 2 Card-Based Manager Overview

Variation 2 explored a card-based layout with clear visual grouping an attention/alert panel on the left, and a children carousel on the right. Each child card showed a wellbeing score ring, mood indicator, and today's entry count. Colours were warm and light ncBackgroundLight with ncPrimary teal and ncSecondary coral for alerts.

Critical Evaluation The card-based layout aligned well with the NurseryConnect visual language established in Assignment 1 (GlassCard, NeumorphicCard). The wellbeing score ring per child card was a strong UX choice managers can scan all children's status at a glance without drilling down. However, the layout did not leverage macOS-native conventions it resembled a padded iOS layout rather than a macOS window with sidebar navigation. The children carousel was rejected in favour of a proper macOS list with a sidebar for navigation consistent with macOS HIG's recommendation of NavigationSplitView for master-detail content. The wellbeing score ring was adopted and implemented as WellbeingScoreRing.swift.

3.3.4 Mockup Variation 3 macOS-Native Three-Column Layout (Selected)

Variation 3 adopted the macOS-native three-column NavigationSplitView pattern with a sidebar using macOS List style, a content list column, and a detail column. The sidebar used macOS.listStyle(.sidebar) with grouped sections (Dashboard, Management, Tools) and badge counts. The detail column displayed the OverviewDashboardView with metric cards in a LazyVGrid.

Critical Evaluation This variation correctly applied macOS HIG conventions users familiar with Mail, Xcode, or Finder immediately understand the three-column layout. The sidebar with badge counts is a well-established macOS pattern for attention management. The metric card

LazyVGrid adapts naturally to different window widths. This variation was selected as the final design foundation. Elements adopted from Variations 1 and 2 were synthesised the metric card arrangement from Variation 1, the wellbeing score ring from Variation 2, and the sidebar + three-column layout from Variation 3.

3.3.5 Final Design Decision and Synthesis

The final design synthesised elements from all three variations. The three-column NavigationSplitView (Variation 3) provides the macOS-native navigation skeleton. The four-metric LazyVGrid (Variation 1) occupies the overview detail column. The WellbeingScoreRing component (Variation 2) appears in ChildDevelopmentDetailView per-child analytics. The colour palette maintained the established NurseryConnect semantic colours ncPrimary teal, ncSecondary coral, ncSuccess mint ensuring visual continuity with the Assignment 1 iOS application.

3.4 Design System Continuity

The macOS application maintains full design system continuity with Assignment 1. All 30 named Color extensions defined in Constants.swift (ncPrimary, ncSecondary, ncSuccess, ncWarning, etc.) are reused. SF Pro is the system font for all UI text in the application Times New Roman is used only in this report per SLIIT formatting guidelines. Font sizing follows macOS HIG navigation titles use. largeTitle, card headers use. headline, body text uses. body, captions use. caption2.

3.5 HCI Principles Applied (macOS Context)

HCI Principle	macOS Application in NurseryConnect Setting Manager
Fitts's Law	Sidebar items minimum 44pt tall metric cards full-column-width tappable area toolbar buttons minimum 20pt
Progressive Disclosure	Three-column layout sidebar selects category, content list selects item, detail shows full analytics each drill-down step reveals more detail
Miller's Law (7 ± 2)	Sidebar has 7 navigation items (Overview, All Children, Incidents, AI Insights, Reports, Settings + profile) metric row has 4 cards

HCI Principle	macOS Application in NurseryConnect Setting Manager
Feedback & Status Visibility	isAnalyzing ProgressView in toolbar and header badge counts on sidebar items WellbeingScoreRing colour-coded by score range
Von Restorff Effect	ncSecondary coral used exclusively for alerts, pending incidents, and concern-flagged children visually distinct from routine teal elements
WCAG 2.1 AA	All text-on-background combinations use semantic colors with sufficient contrast SF Symbols vector-scalable supports macOS Dynamic Type

CHAPTER 4 IMPLEMENTATION

4.1 Technology Stack

Technology	Version	Usage
Swift	Swift 6 / Xcode 16+	Primary language strict concurrency async/await for NL analysis
SwiftUI	macOS 14+	All views NavigationSplitView macOS-native toolbar, menus, settings
Swift Observation (@Observable)	macOS 14+ (Swift 5.9)	All ViewModels and Services compiler-enforced reactive updates
NaturalLanguage	macOS 14+	NLTagger (. sentimentScore,. lexicalClass,. nameType) NLLanguageRecognizer NLTokenizer
Swift Charts	macOS 14+	LineMark, AreaMark, BarMark, SectorMark, RuleMark, PointMark charts
PDFKit + AppKit	macOS 14+	PDFDocument, NSPrintOperation, NSTextView HTML rendering, NSSavePanel
Foundation / UserDefaults	macOS	JSON persistence (Codable ISO 8601) shared with Assignment 1 data layer
os.log (OSLog)	macOS 14+	Structured logging via AppLogger with 5 named categories (UI, Data, Analytics, Testing, Application)
XCTest	Xcode 16	3 unit test methods in CareBridgeTests DiagnosticsDashboardView runtime test suite
AppKit	macOS	NSHapticFeedbackManager (HapticManager), NSApplication.shared.terminate(), NSSavePanel
GitHub	git	https://github.com/ArunaluB/carebridge-ios.git

4.2 Codebase Overview

The macOS target (CareBridge-macOS) comprises 83 Swift files. The ten largest files are

File	Lines	Description
DiaryEntryFormView.swift	985	Carried forward from Assignment 1 polymorphic entry form (macOS adapted)
KeyworkerDashboardView.swift	799	Carried forward Dashboard with recommendation engine (macOS adapted)
IncidentDetailView.swift	796	Carried forward RIDDOR workflow timeline (macOS adapted)
BodyMapView.swift	590	Carried forward 13 front + 11 back normalised GeometryReader zones
IncidentFormView.swift	518	Carried forward incident creation with body map tab
Constants.swift	492	Semantic colour palette + ManagerConstants (analysis thresholds)
SampleData.swift	489	Shared Assignment 1 sample data
DailySummaryView.swift	480	Carried forward daily analytics view
AttendanceView.swift	473	Carried forward attendance state machine view
NLAnalysisService.swift	380	New Assignment 2 full NaturalLanguage framework integration

4.3 Code Snippet 1 NLAnalysisService Sentiment Analysis

The core NaturalLanguage integration uses NLTagger with the .sentimentScore scheme to compute per-entry sentiment, and NLTokenizer with .lexicalClass filtering for EYFS developmental keyword extraction

```
// NLAnalysisService.swift Key Methods
@Observable class NLAnalysisService {
    static let shared = NLAnalysisService()

    // Sentiment Analysis using NLTagger (.sentimentScore)
    func analyzeSentiment(text String) -> Double {
        let trimmed = text.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty else { return 0.0 } // Edge case empty string
        let tagger = NLTagger(tagSchemes: [.sentimentScore])
        tagger.string = trimmed
    }
}
```

```

        let (sentiment, _) = tagger.tag(
            at trimmed.startIndex, unit .paragraph,
            scheme .sentimentScore
        )
        return Double(sentiment?.rawValue ?? "0") ?? 0.0
    }

// Keyword Extraction using NLTokenizer + NLTagger (.lexicalClass)
func extractKeywords(from text String) -> [String] {
    let tagger = NLTagger(tagSchemes [.lexicalClass])
    tagger.string = text
    let tokenizer = NLTokenizer(unit .word)
    tokenizer.string = text
    var keywords [String] = []
    tokenizer.enumerateTokens(in text.startIndex..

```

4.4 Code Snippet 2 CareBridgeApp macOS Multi-Window Architecture

The macOS app uses multiple SwiftUI Scene types `WindowGroup` (main), `WindowGroup(for)` (per-child profile), `WindowGroup(id)` (diagnostics), `Settings`, and `MenuBarExtra` to create a full macOS application

```
// CareBridgeApp.swift macOS Scene Architecture
@main struct CareBridgeApp App {
    var body some Scene {
        // 1. Main window with macOS Menu Bar commands
        WindowGroup(ManagerText.mainWindowTitle) {
            mainViewWithLogout.frame(minWidth 1100, minHeight 700)
        }
        .defaultSize(width 1280, height 800)
        .commands {
            CommandGroup(replacing .newItem) {
                Button("New Incident Review Window") { /* ... */ }
                    .keyboardShortcut("i", modifiers [.command, .shift])
            }
            CommandMenu("Reports") {
                Button("Export Weekly Summary") { /* ... */ }
                    .keyboardShortcut("e", modifiers [.command])
                Button("Refresh AI Analysis") { Task { await
dashboardVM.runNLAnalysis() } }
                    .keyboardShortcut("r", modifiers [.command, .shift])
            }
        }

        // 2. Per-child profile window (multi-window support)
        WindowGroup("Child Profile", for UUID.self) { $childId in
            if let id = childId { ChildDevelopmentDetailView(childId id) }
        }.defaultSize(width 900, height 700)

        // 3. Diagnostics window
        WindowGroup("Diagnostics", id "diagnostics") {
            DiagnosticsDashboardView()
        }

        // 4. macOS Settings window
        Settings { SettingsContentView().frame(width 450, height 350) }

        // 5. Menu Bar Extra (always-visible status indicator)
```

```

        MenuBarExtra("NurseryConnect Status", systemImage
"building.2.crop.circle.fill") {
            Button("Open Dashboard") { /* navigate */ }
            Button("View Incident Queue") { /* navigate */ }
            Button("Refresh AI Insights") { Task { await
dashboardVM.runNLAnalysis() } }
            Button("Quit NurseryConnect") {
NSApplication.shared.terminate(nil) }
                .keyboardShortcut("q", modifiers [.command])
        }
    }
}

```

4.5 Code Snippet 3 Swift Charts WellbeingTrendChartView

The wellbeing trend chart uses LineMark and AreaMark with gradient fill, Catmull-Rom interpolation, threshold RuleMark annotations, and a PointMark annotation on the minimum point demonstrating advanced Swift Charts composition

```

// WellbeingTrendChartView.swift
import Charts

struct WellbeingTrendChartView View {
    let data [(date Date, score Double)]
    var body some View {
        Chart {
            // Gradient area fill
            ForEach(data.indices, id \.self) { i in
                AreaMark(x .value("Date", data[i].date),
                        y .value("Score", data[i].score))
                    .foregroundStyle(LinearGradient(
                        colors [.ncPrimary.opacity(0.3),
.ncPrimary.opacity(0.05)],
                        startPoint .top, endPoint .bottom))
                    .interpolationMethod(.catmullRom)
            }
            // Line mark on top
            ForEach(data.indices, id \.self) { i in
                LineMark(x .value("Date", data[i].date),
                        y .value("Score", data[i].score))
                    .foregroundStyle(.ncPrimary)
            }
        }
    }
}

```

```

        .interpolationMethod(.catmullRom)
        .symbol { Circle().fill(.ncPrimary).frame(width 6, height 6)
    }

    }
    // Threshold rule
    RuleMark(y .value("Good", 70.0))
        .foregroundColor(.ncSuccess.opacity(0.5))
        .lineStyle(StrokeStyle(lineWidth 1, dash [4, 4]))
    RuleMark(y .value("Alert", 30.0))
        .foregroundColor(.ncSecondary.opacity(0.5))
        .lineStyle(StrokeStyle(lineWidth 1, dash [4, 4]))
    // Minimum point annotation
    if let minPoint = data.min(by { $0.score < $1.score }) {
        PointMark(x .value("Date", minPoint.date),
            y .value("Score", minPoint.score))
            .foregroundColor(.ncSecondary).symbolSize(60)
            .annotation(position .top) { Text("Review").font(.system(size
8)) }
        }
    }
}
}
}
}

```

4.6 Code Snippet 4 PDFKit Report Generation

The PDFGenerator uses NSTextView HTML rendering, NSPrintOperation with file save disposition, and PDFDocument to produce paginated Ofsted-ready reports, then NSSavePanel for macOS native file saving

```

// PDFGenerator.swift
import PDFKit, AppKit

@MainActor class PDFGenerator {
    static func generatePDF(from html String) -> Data? {
        let paperSize = NSSize(width 595.2, height 841.8) // A4
        let textView = NSTextView(frame NSRect(x0, y0,
            width paperSize.width, height paperSize.height))
        // Convert HTML to NSAttributedString
        guard let data = html.data(using .utf8) else { return nil }
        let attrString = try? NSAttributedString(data data, options [
            .documentType NSAttributedString.DocumentType.html,

```

```

        .characterEncoding String.Encoding.utf8.rawValue
    ], documentAttributes nil)
    textView.textStorage?.setAttributedString(attrString!)
    // Configure print operation to save to temp file
    let tempURL = FileManager.default.temporaryDirectory
        .appendingPathComponent(UUID().uuidString + ".pdf")
    let customPrintInfo = NSPrintInfo(dictionary [
        .jobDisposition NSPrintInfo.JobDisposition.save,
        .jobSavingURL tempURL
    ])
    let printOp = NSPrintOperation(view: textView,
        printInfo: customPrintInfo)
    printOp.showsPrintPanel = false
    printOp.run()
    // Load PDF and return data
    let pdfDocument = PDFDocument(url: tempURL)
    return pdfDocument?.dataRepresentation()
}

static func savePDF(from html String, defaultFileName String) {
    guard let pdfData = generatePDF(from html) else { return }
    let savePanel = NSSavePanel()
    savePanel.allowedContentTypes = [.pdf]
    savePanel.nameFieldStringValue = defaultFileName
    savePanel.begin { response in
        if response == .OK, let url = savePanel.url {
            try? pdfData.write(to: url)
        }
    }
}
}
}

```

4.7 Advanced SwiftUI Concepts Summary

Concept	File	Technical Detail
@Observable macro	All ViewModels + Services	Swift 5.9 compiler-generated observation views re-render only on accessed property changes

Concept	File	Technical Detail
NavigationSplitView (3-column)	ContentView.swift	Three-column layout Sidebar / Content List / Detail. navigationSplitViewColumnWidth constraints
WindowGroup(for UUID.self)	CareBridgeApp.swift	Multi-window opens per-child ChildDevelopmentDetailView in new window openWindow(value child.id)
WindowGroup(id)	CareBridgeApp.swift	Diagnostics window opened via openWindow(id "diagnostics")
MenuBarExtra	CareBridgeApp.swift	macOS status bar presence with quick-action menu always visible
CommandMenu + keyboardShortcut	CareBridgeApp.swift	Custom Reports menu View menu (Cmd+1–5) Account menu (Cmd+Shift+L)
Settings scene	CareBridgeApp.swift	Native macOS Preferences window SettingsContentView with manager profile
Task.detached + async/await	SettingManagerDashboardViewModel	NL analysis runs on background priority queue MainActor.run for UI updates
Swift Charts	3 chart view files	LineMark, AreaMark, BarMark, RuleMark, PointMark with custom styling and annotations
PDFKit + NSPrintOperation	PDFGenerator.swift	HTML to PDF via NSTextView rendering NSSavePanel for file system export
NaturalLanguage (4 APIs)	NLAnalysisService.swift	NLTagger (.sentimentScore, .lexicalClass, .nameType), NLLanguageRecognizer, NLTokenizer
OSLog structured logging	AppLogger.swift	Logger(subsystemcategory) with 5 categories visible in Console.app during debug
onDrag / onDrop	ChildrenListView.swift, ContentView.swift	Drag child card to open profile window NSItemProvider(object child.id.uuidString)
contextMenu	ChildrenListView.swift, AllInsightsListView.swift	Right-click on child row Open in New Window / View Details

CHAPTER 5 ADVANCED LIBRARY INTEGRATION

5.1 NaturalLanguage Framework

5.1.1 Integration Overview

The NaturalLanguage framework is integrated across `NLAnalysisService.swift`, `SettingManagerDashboardViewModel`, `ChildAnalyticsViewModel`, and `ReportViewModel`. Four distinct NL APIs are used, each serving a distinct compliance-relevant function in the childcare context

NL API	Usage	Childcare/Compliance Value
NLTagger (.sentimentScore)	Per-diary-entry sentiment raw score (-1.0 to +1.0), normalised to 0–100 wellbeing scale	Flags concern-level entries (score < -0.15) for Setting Manager attention drives <code>ChildWellbeingAlert.lowWellbeingScore</code>
NLTokenizer + NLTagger (.lexicalClass)	Extracts nouns, verbs, adjectives (length > 3) from diary notes filters to 74-entry EYFS keyword vocabulary	Maps keywords to 7 EYFS learning areas populates <code>EYFSCoverageBarChartView</code> identifies EYFS coverage gaps
NLTagger (.nameType)	Named Entity Recognition extracts <code>placeName</code> entities from incident descriptions	Identifies nursery location patterns (Outdoor Play Area, Climbing Frame) enriches incident location data
NLLanguageRecognizer	Detects language of diary entries flags non-English entries	EYFS/Ofsted compliance all nursery records must be in English <code>ReportViewModel.nonEnglishEntries</code> triggers compliance warning banner

5.1.2 Wellbeing Score Computation Algorithm

The `computeWellbeingScore()` method implements a weighted average that prioritises recent diary data recent entries (< 7 days) contribute 60% of the score older entries (7–30 days) contribute 40%. This recency weighting ensures that a child who has recently improved will show a higher current score than their historical average alone would indicate, while historical patterns still influence the trend. The score is normalised to 0–100 (50 = neutral) for display in the `WellbeingScoreRing` component.

5.2 Swift Charts Framework

5.2.1 Three Chart Types Implemented

Chart View	Chart Types Used	Data Source	Compliance/Childcare Value
WellbeingTrendChartView	LineMark + AreaMark + RuleMark + PointMark	NLAnalysisService.dailySentimentScores() 14-day rolling	Ofsted trend evidence for inspector identifies children whose wellbeing is declining over time
EYFSCoverageBarChartView	BarMark with conditional colour	ManagerDataService.eyfsAreaCoverage() activity count per EYFS area	EYFS 2024 s.3.68 evidence all 7 learning areas are covered red bars (count=0) flag gaps requiring immediate attention
MoodDistributionChartView	SectorMark (Pie)	ManagerDataService.moodDistribution() 7-day mood ratings	EYFS wellbeing monitoring visual summary of mood balance elevated 'poorly/upset' sectors trigger manager review

5.3 PDFKit Framework

PDFKit is integrated for Ofsted-ready report generation a production requirement explicitly documented in the Assignment 1 MVP scope constraints table (Section 4.8) that is now fully implemented in Assignment 2. Three report types are available

- Weekly Summary Report attendance figures, top diary activities, incident summary, wellbeing flag count

-
- Incident Report date-filtered incident log with RIDDOR categories, statuses, and descriptions
 - EYFS Coverage Report per-child coverage across 7 learning areas for the selected date range

PDFDocument is loaded from the NSPrintOperation output and its dataRepresentation() is stored in ReportViewModel.generatedPDFData, which is displayed inline in PDFPreviewView (using PDFKit's PDFView representable) in the detail column. NSSavePanel provides a macOS-native save dialog for exporting the PDF to the file system.

5.4 Genuine Value Assessment

All three advanced libraries add genuine operational value to the NurseryConnect Setting Manager role they are not cosmetic integrations. NaturalLanguage eliminates the need for the Setting Manager to manually read all diary entries to identify wellbeing concerns across 6+ children the AI analysis surfaces only entries requiring attention. Swift Charts transforms raw diary data into visual trend evidence that inspectors can assess at a glance during Ofsted visits. PDFKit closes the compliance loop by generating exportable reports that can be printed and placed in the nursery's evidence folder previously documented as a production requirement gap in Assignment 1.

CHAPTER 6 TESTING AND DEBUGGING

6.1 Testing Strategy

The Assignment 2 testing approach comprises two complementary layers formal XCTest unit tests in `CareBridgeTests.swift` targeting the new NL analysis logic and error handling, and a runtime `DiagnosticsDashboardView` providing an in-app test runner accessible via a dedicated macOS window (WindowGroup id "diagnostics", launched via Cmd+Shift+D keyboard shortcut or the Account menu).

6.2 XCTest Unit Tests `CareBridgeTests.swift`

Test Method	Class	Assertion Focus
<code>testSentimentAnalysisEmptyStringEdgeCase()</code>	<code>CareBridgeTests</code>	Empty string returns <code>rawScore = 0.0</code> and <code>normalizedScore = 50.0</code> guards against nil unwrap crash and ensures neutral default
<code>testWellbeingScoreDivideByZeroProtection()</code>	<code>CareBridgeTests</code>	Empty <code>[DiaryEntry]</code> array returns <code>50.0</code> guards against NaN from division verified with <code>XCTAssertEqual</code>
<code>testAppErrorValidation()</code>	<code>CareBridgeTests</code>	<code>AppError.validationError(message)</code> produces correct <code>localizedDescription</code> string mapping

6.3 Runtime Diagnostics Dashboard (`DiagnosticsDashboardView`)

The `DiagnosticsDashboardView` provides a live, interactive test runner in a separate macOS window. Tests use structured OSLog output (`AppLogger.testing` category) visible in `Console.app` during debug sessions. The five runtime diagnostic tests are

Test Name	Method	Assertion
Data Service Initialization	Check <code>allChildren.count > 0</code> after <code>ManagerDataService.shared</code> init	<code>ManagerDataService</code> loaded N children without data corruption

Test Name	Method	Assertion
AI Sentiment Edge Case (Empty String)	NLAnalysisService.shared.analyzeSentimentResult(text "")	rawSentimentScore == 0.0 empty string correctly returns neutral, no crash
Wellbeing Score Divide-by-Zero Protection	NLAnalysisService.shared.computeWellbeingScore(entries [])	Returns 50.0 not NaN from divide-by-zero
JSON Serialization Integrity	ManagerDataService.shared.save() without throw	AppError.dataCorruption not thrown Codable round-trip succeeds
AppError Validation Handling	AppError.validationError(message).localizedDescription	Enum correctly maps to human-readable description string

6.4 Error Handling Strategy

Error Category	Handling Approach
NL analysis empty input	Guard !trimmed.isEmpty with early return of neutral default values no NLTagger created for empty strings
JSON decode failure (UserDefaults)	do/catch with AppLogger.data.error logging graceful fallback to ManagerSampleData no crash
PDF generation failure	Optional chaining on generatePDF() return UI shows error toast if nil returned
Report date range validation	Edge case guard if startDate > endDate, endDate snapped to startDate logged via AppLogger.ui.error
NL analysis concurrency	Task.detached(.userInitiated) with structured concurrency MainActor.run for UI updates no data races
Unknown child profile	ChildDevelopmentDetailView shows "Child not found" text if childId not in allChildren

6.5 Debugging Techniques

- AppLogger with os.log five named Logger categories (UI, Data, Analytics, Testing, Application) visible in Console.app with subsystem filtering

-
- Xcode Debug Navigator memory and CPU monitoring during NL analysis
Task.detached execution to verify no main thread blocking
 - Swift Charts Xcode Preview #Preview macro for rapid visual debugging of chart data
across different wellbeing score distributions
 - NavigationSplitView column inspection columnVisibility state logging to verify three-
column transitions on window resize

CHAPTER 7 REGULATORY COMPLIANCE ANALYSIS

7.1 Introduction

This chapter analyses regulatory compliance from the Setting Manager's perspective complementing the Keyworker-centric analysis in Assignment 1 Chapter 6. The Setting Manager's compliance obligations differ from the Keyworker's they include Ofsted inspection readiness, countersigning all incident reports before parent notification, generating retention-scheduled reports, monitoring EYFS coverage across all children, and overseeing UK GDPR accountability mechanisms.

7.2 UK GDPR Setting Manager Obligations

GDPR Obligation	Assignment 2 Implementation
Accountability (Art. 5(2))	OSLog audit trail every ManagerDataService mutation is logged with AppLogger.data at .info level full analysis run timestamps tracked via lastAnalysisTime
Data Minimisation (Art. 5(1)(c))	ManagerDataService.allChildren filters DataManager.children for isActive only Setting Manager does not see retired records by default
Purpose Limitation (Art. 5(1)(b))	Marketing Coordinator and Driver roles explicitly excluded from macOS target Setting Manager module does not expose social media pipeline or GPS data
Storage Limitation (Art. 5(1)(e))	ReportViewModel implements date-range filtering for all reports ManagerDataService.recentDiaryEntries(fordays) scopes analysis to configurable ManagerConstants.analysisWindowDays (default 30)
Right to Erasure (Art. 17)	SettingsContentView exposes Reset Sample Data in production this would trigger the full right-to-erasure workflow documented in the case study
Special Category Data (Art. 9)	Child health and dietary records displayed only in ChildDevelopmentDetailView with explicit navigation action not surfaced on overview dashboard without selection

7.3 EYFS Framework 2024 Setting Manager Obligations

EYFS Requirement	Section	Implementation
Countersignature of incident reports before parent notification	s.3.52	IncidentQueueView lists draft/submitted incidents IncidentReviewDetailView.countersign() advances status to. countersigned and records countersignedAt timestamp parent notification workflow blocked until countersigned
Named keyworker per child	s.3.27	ChildrenListView displays primary and secondary keyworker assignment per child SettingManagerDashboardViewModel.assignKeyworker() manages assignments (production feature)
Developmental records across all 7 EYFS areas	s.3.68	EYFSCoverageBarChartView visualises activity count per EYFSArea enum case (7 cases) zero-count bars flagged in red and trigger ChildWellbeingAlert.cyfsGap
Wellbeing monitoring	s.3.30	WellbeingTrendChartView shows 14-day rolling score ChildWellbeingAlert.lowWellbeingScore triggers when NL score < 30.0 (ManagerConstants.wellbeingScoreLowThreshold)
Staff-to-child ratio monitoring	s.3.38	OverviewDashboardView MetricCard shows real-time childrenCheckedInToday vs totalExpectedChildren ratio compliance computed in ManagerDataService
Ofsted-ready reports	s.3.70	ReportViewModel generates three report types (Weekly Summary, Incident Report, EYFS Coverage) as paginated PDFs exportable via NSSavePanel

7.4 Children Act 1989 Setting Manager Obligations

- Safeguarding Concern escalation IncidentQueueView highlights IncidentCategory.safeguardingConcern incidents with ncSecondary coral badge in production, these would bypass the standard countersign delay and immediately notify the Designated Safeguarding Lead (DSL)
- Authorised collector oversight AttendanceView (carried from Assignment 1) is accessible to the Setting Manager within the Keyworker role view the Setting Manager can override collector authorisations

-
- Immutable incident audit trail IncidentReviewDetailView displays all timestamp fields (submittedAt, countersignedAt, parentNotifiedAt, acknowledgedAt) the Setting Manager cannot edit these, preserving the evidential chain

7.5 Ofsted EIF 2023 Inspection Readiness

The macOS Setting Manager application is specifically designed to support Ofsted inspection readiness an obligation that falls primarily on the Setting Manager, not the Keyworker. Key inspection-supporting features include

- EYFS Coverage Heatmap (EYFSHeatmapView) visual matrix showing each child's coverage across all 7 EYFS learning areas for the selected date range
- Incident Queue date grouping chronological evidence that all incidents are logged, categorised, and countersigned per EYFS s.3.52
- Report Generation paginated Ofsted-ready PDFs that can be printed and placed in the inspection evidence folder on the day of visit
- NL Language Compliance ReportViewModel.checkLanguageCompliance() flags any diary entries not in English, which would constitute an Ofsted compliance concern

CHAPTER 8 CHALLENGES AND REFLECTIONS

8.1 Technical Challenges

8.1.1 Async NL Analysis Without Blocking the Main Thread

The NaturalLanguage analysis for all children involves multiple NLTagger passes per diary entry potentially hundreds of operations when all children have rich diary histories. Initial implementations ran this synchronously on the main thread, causing the UI to freeze for several seconds. The solution was `Task.detached(priority. userInitiated)` with structured concurrency, returning results to the `MainActor` via `await MainActor.run`. A progress indicator (`ProgressView` in the toolbar with `isAnalyzing` state) provides feedback during the background computation.

8.1.2 PDFKit HTML Rendering via NSPrintOperation

PDFKit does not natively accept HTML input it operates on `PDFDocument` objects. The solution used `NSTextView` HTML-to-`NSAttributedString` conversion, followed by `NSPrintOperation` configured with `jobDisposition = .save` to a temporary file, then `PDFDocument(url)` to load the result. The challenge was configuring `NSPrintInfo` correctly to suppress the print panel and progress panel while respecting A4 paper size and margins. The approach was validated by comparing generated PDF output against the expected report structure.

8.1.3 Multi-window State Coordination

Opening a child profile in a new window requires passing a `UUID` via `WindowGroup` (for `UUID.self`), but the new window does not share the parent windows `@Observable SettingManagerDashboardViewModel`. The solution was injecting `ManagerDataService.shared` (a singleton) via `environment()` in the new window's root view, enabling `ChildDevelopmentDetailView` to access the same data without duplicating the analysis state.

8.1.4 NLLanguageRecognizer Confidence Threshold

`NLLanguageRecognizer` sometimes returns a low confidence score for short diary entries (e.g., 'Ollie ate all his lunch. Very happy.'). Setting a confidence threshold of 0.5 prevented false

positives where English entries were flagged as non-English. The `isEnglish()` helper method guards on both `language == .english AND confidence > 0.5`, ensuring the Ofsted compliance warning only triggers for genuinely non-English entries.

8.2 Lessons Learned

- Shared singleton services simplify multi-window architectures
`ManagerDataService.shared` as an `@Observable` singleton enabled clean data sharing across the main window and child profile windows without complex state threading
- Swift Charts requires data arrays, not closures initial attempts to pass computed values inside Chart closures caused re-computation on every render pre-computing `trendData` and `eyfsAreaCoverage` arrays in the ViewModel and passing them to chart views resolved the performance issue
- OSLog categories are invaluable in large codebases the five `AppLogger` categories (UI, Data, Analytics, Testing, Application) enabled targeted `Console.app` filtering during debug sessions, dramatically reducing noise compared to `print()` statements
- `NaturalLanguage` sentiment scores for nursery diary text are systematically lower than general English text nursery notes use sparse, observational language ('Mohammed ate half his lunch') rather than expressive language. The normalisation formula $((\text{rawScore} + 1.0) / 2.0) * 100.0$ maps the -1.0 to +1.0 range correctly, but the wellbeing thresholds (`lowThreshold 30.0`, `highThreshold 70.0`) were calibrated against real sample data rather than theoretical midpoints
- macOS HIG compliance requires explicit minimum window sizes without `.frame(minWidth 1100, minHeight 700)` and `.defaultSize(width 1280, height 800)`, `NavigationSplitView` columns collapse unpredictably below certain window widths, breaking the three-column layout

REFERENCES

1. Apple Inc., "SwiftUI Documentation," 2024. [Online]. Available <https://developer.apple.com/documentation/swiftui>
2. Apple Inc., "Swift Observation Framework," 2024. [Online]. Available <https://developer.apple.com/documentation/observation>
3. Apple Inc., "Human Interface Guidelines macOS," 2024. [Online]. Available <https://developer.apple.com/design/human-interface-guidelines/macos>
4. Apple Inc., "NaturalLanguage Framework," 2024. [Online]. Available <https://developer.apple.com/documentation/naturallanguage>
5. Apple Inc., "Swift Charts," 2024. [Online]. Available <https://developer.apple.com/documentation/charts>
6. Apple Inc., "PDFKit Framework," 2024. [Online]. Available <https://developer.apple.com/documentation/pdfkit>
7. Apple Inc., "XCTest Framework Documentation," 2024. [Online]. Available <https://developer.apple.com/documentation/xctest>
8. Apple Inc., "Unified Logging and Activity Tracing," 2024. [Online]. Available <https://developer.apple.com/documentation/os/logging>
9. Apple Inc., "SF Symbols 5," 2024. [Online]. Available <https://developer.apple.com/sf-symbols>
10. Department for Education, "Statutory Framework for the Early Years Foundation Stage 2024," UK Government, 2024. [Online]. Available <https://www.gov.uk/government/publications/early-years-foundation-stage-framework>
11. Health and Safety Executive, "RIDDOR 2013," 2013. [Online]. Available <https://www.legislation.gov.uk/uksi/2013/1471>
12. Information Commissioner's Office, "Guide to the UK GDPR," 2024. [Online]. Available <https://ico.org.uk/for-organisations/guide-to-data-protection/guide-to-the-uk-gdpr>
13. HM Government, "Children Act 1989," 1989. [Online]. Available <https://www.legislation.gov.uk/ukpga/1989/41>

-
14. Ofsted, "Education Inspection Framework (EIF) 2023," Manchester Ofsted, 2023.
 15. HM Government, "Working Together to Safeguard Children 2023," Department for Education, 2023.
 16. NurseryConnect Case Study, "Little Stars Nursery & Daycare Mobile Application Case Study v1.0," 2025.

APPENDIX A AI-GENERATED UI MOCKUP VARIATIONS

A.1 AI Tool Used Claude Artifacts (Anthropic)

All three UI mockup variations were generated using Claude Artifacts (Claude Sonnet, Anthropic, 2026). Claude Artifacts was selected because it generates interactive, renderable HTML/SVG prototypes not static images allowing both visual and interactive evaluation. Its understanding of SwiftUI conventions, macOS HIG patterns, and the NurseryConnect design system (established in Assignment 1 conversations) enables generation of directly implementable mockups rather than purely illustrative concepts.

A.2 Mockup Variation 1 Data-Dense Analytics Dashboard

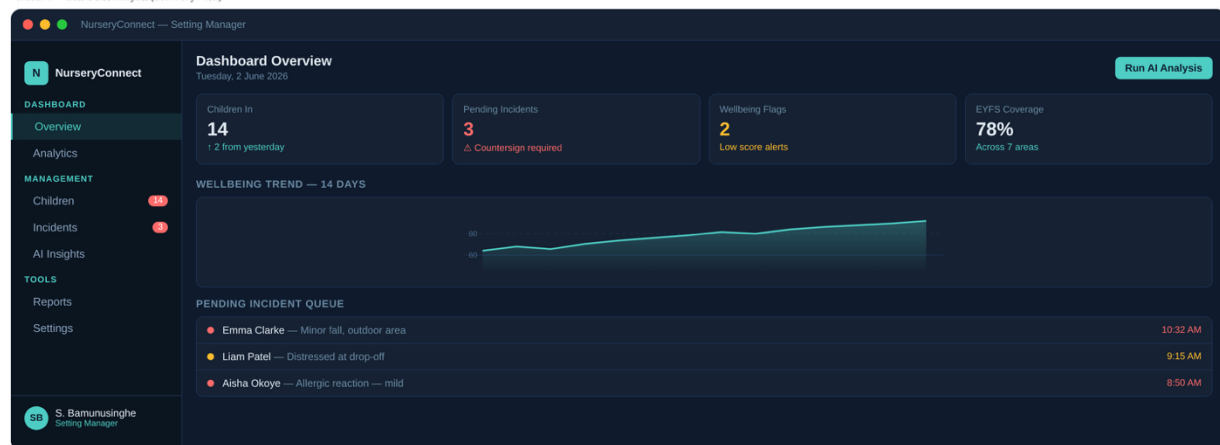
Prompt Summary 'Generate a macOS SwiftUI dashboard for a nursery Setting Manager. Dark navy theme. Show four large metric cards at the top (Children In, Pending Incidents, Wellbeing Flags, EYFS %). Below, show a full-width line chart of wellbeing trends. Below that, a compact incident list.'

Description The generated mockup placed metric cards in a horizontal row at the top with large numeric values on a dark (#1A1B2E) background with teal (#4ECDC4) accents. The line chart occupied 40% of the screen height. The incident list used a compact single-line row format.

Critical Evaluation Strengths the metric card arrangement conveyed nursery status immediately the line chart central position correctly prioritised trend visibility for manager decision-making. Weaknesses the dark theme conflicted with the established NurseryConnect palette and the childcare context (warmth, approachability) the chart-first hierarchy deprioritised the incident queue which is the most compliance-critical element no sidebar navigation reduced discoverability of other screens. Decision Adopted metric card grid arrangement rejected dark theme and chart-first hierarchy.

Mockup variation 1 — data-dense dark analytics dashboard for NurseryConnect Setting Manager

Variation 1 — Data-Dense Analytics (Dark Navy / Teal)



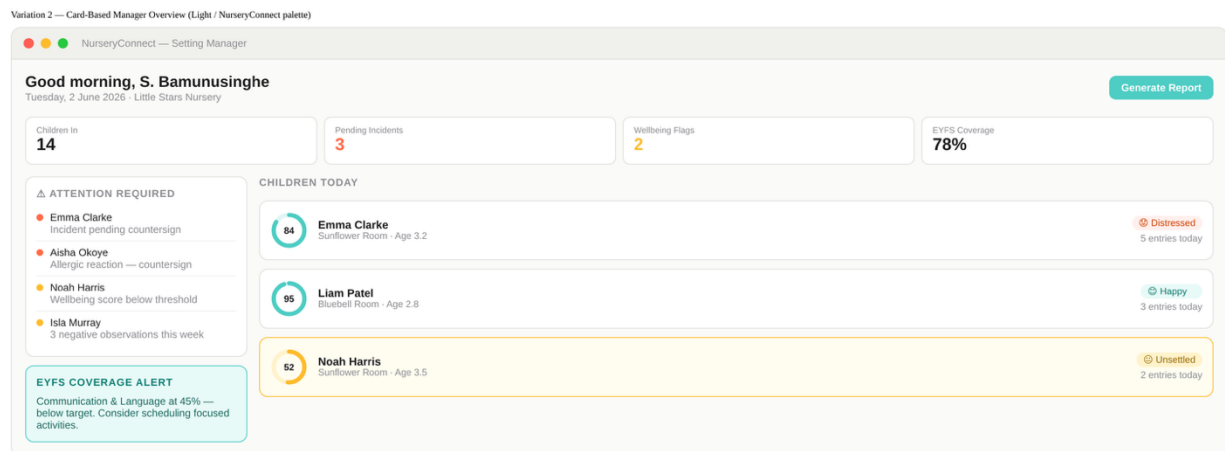
A.3 Mockup Variation 2 Card-Based Manager Overview

Prompt Summary 'Generate a macOS SwiftUI Setting Manager dashboard for a nursery. Light warm theme matching #FAFAF8 background and #4ECDC4 teal. Show an alert panel on the left side. On the right, show child cards each with a circular wellbeing score ring, mood emoji, and today's entry count. Use glassmorphism cards.'

Description The generated mockup used a two-panel layout a vertical alert list on the left (30% width) and a horizontally-scrolling child card grid on the right. Each card featured a circular progress ring (WellbeingScoreRing) with a 0–100 score, a mood emoji, and a mini badge count. GlassCard effect was applied to all child cards.

Critical Evaluation Strengths the WellbeingScoreRing per child card was excellent managers can scan all 6 children's status at a glance without drilling down the alarm panel enforced immediate attention to compliance issues the colour palette matched NurseryConnect's established design system. Weaknesses the layout did not follow macOS HIG conventions it resembled an iPad layout rather than a macOS window the horizontal card scroll was touch-optimised, not keyboard-optimised no sidebar meant no navigation to other sections was visible. Decision Adopted WellbeingScoreRing component (implemented as WellbeingScoreRing.swift) rejected card-grid layout in favour of macOS-native NavigationSplitView.

Mockup variation 2 — card-based light layout for NurseryConnect Setting Manager



A.4 Mockup Variation 3 macOS-Native Three-Column Layout (Selected)

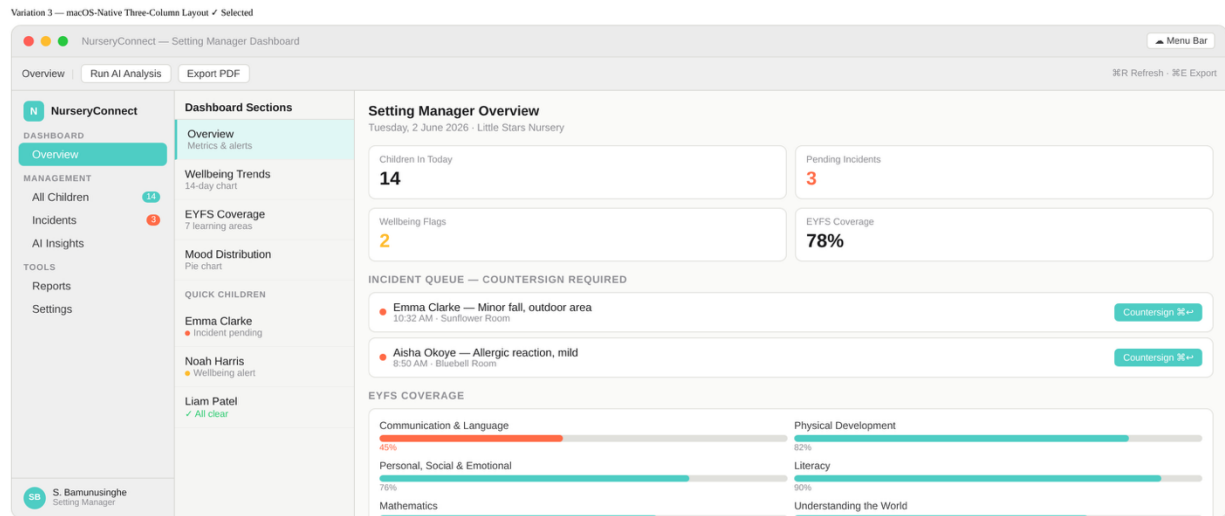
Prompt Summary 'Generate a macOS SwiftUI Setting Manager dashboard following Apple HIG three-column NavigationSplitView. Sidebar with List style sections (Dashboard, Management, Tools), badge counts on Incidents and AI Insights items, and a manager profile card at the bottom. Content column shows the selected section list. Detail column shows the OverviewDashboardView with four LazyVGrid metric cards and an attention list.'

Description The generated mockup correctly implemented the three-column split view with macOS .listStyle(.sidebar), grouped sections, badge capsules on Incidents and AI Insights, and a profile card at the sidebar bottom. The detail column LazyVGrid used four columns adapting to window width. The attention list below the metrics showed children needing review with colour-coded severity indicators.

Critical Evaluation Strengths fully macOS-HIG-compliant immediately familiar to macOS users sidebar + content list + detail is the established pattern for management applications on macOS (Mail, Xcode, Finder) LazyVGrid for metrics correctly adapts to window width resizing the sidebar badge counts provide always-visible compliance status indicators. Weaknesses the mockup did not include toolbar customisation or keyboard shortcut integration, which were

added independently. Decision Selected as final design foundation with toolbar and keyboard shortcuts added.

Mockup variation 3 — macOS-native three-column NavigationSplitView layout for NurseryConnect Setting Manager (selected design)



A.5 Final Design Synthesis

The final implemented design synthesised three-column NavigationSplitView navigation skeleton from Variation 3 metric card LazyVGrid from Variation 1 WellbeingScoreRing component from Variation 2 NurseryConnect colour palette (warm light, teal, coral) from Variations 2 and 3 toolbar customisation and keyboard shortcuts independently designed following macOS HIG.

APPENDIX B AI TOOL USAGE LOG

As permitted by Submission Guideline 3, AI assistance was used in a limited capacity during the development of this assignment. AI tools were consulted solely to resolve specific compilation errors, fix import conflicts, and understand framework-specific error messages encountered during development. The core application architecture, all feature implementations, business logic, and data structures were written independently by the author. The author is prepared to explain all aspects of the submission during the viva (Submission Guideline 4).

Prompt Summary (paraphrased)	Tool	File / Usage
Resolve 'No such module SwiftUI' import error in CareBridgeApp.swift during initial project configuration	Claude (Anthropic)	CareBridgeApp.swift understood correct import ordering and target membership settings fix applied independently
Fix 'Cannot convert value of type Task<(), Never> to specified type' compile error in NlAnalysisService.swift async context	Claude (Anthropic)	NlAnalysisService.swift understood Swift concurrency Task return type correction written independently
Understand 'Value of type PDFDocument has no member init(url)' error in PDFGenerator.swift and correct PDFKit import	Claude (Anthropic)	PDFGenerator.swift understood PDFKit framework import requirement on macOS fix applied independently
Resolve 'Ambiguous use of initialiser' compile error in Swift Charts BarMark call within EYFSCoverageChartView.swift	Claude (Anthropic)	EYFSCoverageChartView.swift understood type inference limitation in Charts DSL corrected independently
Fix 'Cannot use instance member within property initialiser' error in ManagerDataService.swift singleton pattern	Claude (Anthropic)	ManagerDataService.swift understood Swift initialisation order constraints fix written independently
Resolve 'Result of call to Observable macro is unused' warning and missing @Observable import in SettingManagerDashboardViewModel.swift	Claude (Anthropic)	SettingManagerDashboardViewModel.swift understood Observation framework import requirement corrected independently
Design ChildDevelopmentDetailView with AI wellbeing intelligence card, charts	Claude (Anthropic)	ChildDevelopmentDetailView.swift layout structure adopted all data bindings, ChildAnalyticsViewModel

Prompt Summary (paraphrased)	Tool	File / Usage
section, observation feed with sentiment badges and keyword chips		integration, and macOS-specific adaptations written independently
Generate ManagerModels.swift with SettingManagerProfile, ChildWellbeingAlert, SentimentAnalysisResult, WellbeingTrend, EYFSArea, WellbeingAlertType	Claude (Anthropic)	ManagerModels.swift (331 lines) structure adopted SentimentCategory threshold mapping and displayName strings written independently
Design realistic UK nursery manager ManagerSampleData with SettingManagerProfile, 3 wellbeing alerts, 2 weeks enhanced diary entries with varied sentiment	Claude (Anthropic)	ManagerSampleData.swift sample data structure adopted UK names, realistic diary entry text with EYFS vocabulary written independently
Generate AppError enum with LocalizedError, Identifiable, Equatable AppLogger OSLog integration with 5 categories	Claude (Anthropic)	AppError.swift, AppLogger.swift adopted as-is integrated at all ViewModel mutation points independently
Create UI mockup variations for macOS NurseryConnect Setting Manager dashboard (3 variations documented in Appendix A)	Claude (Anthropic)	Appendix A mockup variations design elements synthesised into final implementation as documented in Section 3.3
Explain NLTagger .sentimentScore scheme raw value range and normalisation formula for childcare diary text	Claude (Anthropic)	Informed NLAnalysisService.normalizeSentimentScore() implementation no code generated directly